New chat

L10_S2.pdf
PDF

> "Generate a lecture scribe intended to serve as reference material for exam preparation. Use *ONLY* the provided context (Lecture 5 slides).
> *Instructions:*
> 1. *Purpose:* The scribe must be exam-ready, strictly following the content of the slides.
> 2. *Content:* Include definitions, assumptions, theorem statements, proofs or derivations, and worked examples *exactly* as presented in the lecture materials.
> 3. *Constraints:* Do not introduce new examples, outside information, or hallucinations. Do not use phrases like 'for example' to add your own content.
> 4. *Structure:* Organize the scribe clearly with appropriate headings corresponding to the lecture flow.
>
>
>
> *Goal:* Produce a clean, accurate revision document based solely on the attached file."

Below is a **clean, exam-ready lecture scribe** prepared **strictly from the provided Lecture 10 slides**.
No external material, no added examples, and no additional interpretations have been introduced.
The structure and wording follow the lecture flow and slide content as closely as possible.

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

---

1. Min-Cut Problem

1.1 Why Use Min-Cut?

The min-cut algorithm is used in various applications to solve problems related to network connectivity, reliability, and optimization.

- **Network Design:**
  Min-cut helps in improving the efficiency of communication and optimizing network flow. The algorithm is used in network design to find the minimum capacity cut.
- **Communication Networks:**
  For understanding the vulnerability of networks to failures, minimum cut can be useful. It helps in building robust and fault-tolerant communication networks.
- **VLSI Design:**
  In Very Large Scale Integration (VLSI) design, the algorithm is useful for partitioning circuits into smaller components, leading to reduced interconnectivity complexity.

---

2. What is Min-Cut?

A **cut-set** in a graph is a set of edges whose removal breaks the graph into two or more connected components.

Given a graph

$$G = (V, E)$$

with n vertices, the **minimum cut (min-cut)** problem is to find a **minimum cardinality cut-set** in G.

Min-cut algorithms, such as Karger's algorithm, are random and can be sensitive to the initial choice of edges. If the algorithm happens to contract critical edges early in the process, it might find a smaller cut.

---

3. Edge Contraction

The main operation in the algorithm is **edge contraction**, which is an operation that removes an edge from a graph while simultaneously merging the two vertices.

In contracting an edge (u, v):

- The two vertices u and v are merged into one vertex.
- All edges connecting u and v are eliminated.
- All other edges in the graph are retained.
- The new graph may have **parallel edges** but **no self-loops**.

---

4. Successful Min-Cut Run

A **successful min-cut run** refers to the success in the outcome of an algorithm designed to find the minimum cut in a graph.

(Refer to the figure labeled *Successful Run* in the slides.)

---

5. Unsuccessful Min-Cut Run

An **unsuccessful min-cut run** refers to an iteration of a min-cut algorithm where the algorithm **fails to correctly identify the minimum cut** of a given graph.

(Refer to the figure labeled *Unsuccessful Run* in the slides.)

---

6. Max-Flow Min-Cut Theorem

Theorem Statement

The **max-flow min-cut theorem** states that:

> *In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut.*

Definitions

- **Capacity of a cut:**
  The sum of the capacities of the edges in the cut that are oriented from a vertex $\in X$ to a vertex $\in Y$.
- **Minimum cut:**
  The cut in the network that has the smallest possible capacity.
- **Minimum cut capacity:**
  The capacity of the minimum cut.

- **Maximum flow:**
  The largest possible flow from source S to sink T.

---

7. Deterministic Min-Cut Algorithm

Stoer–Wagner Min-Cut Algorithm

Let s and t be two vertices of a graph G.
Let $G/\{s, t\}$ be the graph obtained by merging s and t.

A **minimum cut of** G can be obtained by taking the smaller of:

- A minimum s–t cut of G, and
- A minimum cut of $G/\{s, t\}$.

Justification

- If there exists a minimum cut of G that separates s and t, then a minimum s–t cut of G is a minimum cut of G.
- Otherwise, a minimum cut of $G/\{s, t\}$ gives the minimum cut.

---

8. Pseudocode: Deterministic Min-Cut

Algorithm 1: MinimumCutPhase(G, a)

1. $A \leftarrow \{a\}$
2. While $A \neq V$:
   - Add to A the most tightly connected vertex.
3. Return the cut weight as the *cut of the phase*.

Algorithm 2: MinimumCut(G)

1. While $|V| \geqslant 1$:
   - Choose any $a \in V$.
   - Call MinimumCutPhase(G, a).
   - If the cut-of-the-phase is lighter than the current minimum cut, store it.
   - Shrink G by merging the two vertices added last.
2. Return the minimum cut.

---

9. Randomized Min-Cut Algorithm

9.1 Why Randomized Algorithms?

Randomized algorithms provide a **probabilistic guarantee of success**.
They provide a more accurate estimate of the minimum cut with fewer iterations.

Additional characteristics:

- Efficiency
- Parallelization
- Approximation guarantees
- Avoidance of worst-case instances
- Heuristic nature
- Robustness

---

10. Karger's Randomized Algorithm

Karger's algorithm repeatedly contracts randomly chosen edges until only two vertices remain. The remaining edges between these two vertices represent a cut.

(Refer to the illustrated contraction sequence in the slides.)

---

11. Pseudocode: Randomized Min-Cut

Algorithm 3: Recursive-Randomized-Min-Cut(G, α)

Input:
An undirected multigraph G with n vertices and an integer constant $\alpha > 0$.

Output:
A cut C of G.

1. If $n \leqslant 3$:
   - $C \leftarrow$ a min-cut of G found using brute force (exhaustive) search.
2. Else:
   - For $i = 1$ to α:
     - $G' \leftarrow$ multigraph obtained by applying

$$n - \lceil \frac{n}{\sqrt{\alpha}} \rceil$$

random contraction steps on G.

- $C' \leftarrow$ Recursive-Randomized-Min-Cut(G', α).
- If $i = 1$ or $|C'| < |C|$, then $C \leftarrow C'$.

3. Return C.

---

12. Comparison: Deterministic vs Randomized Min-Cut

The choice of approach depends on the specific problem.

Deterministic Min-Cut

- Always guarantees an exact minimum cut.
- May have higher time complexity for large graphs.
- Stoer–Wagner algorithm time complexity:

$$O(V \cdot E + V^2 \log V)$$

Randomized Min-Cut

- Provides an approximate minimum cut with high probability.
- Karger's algorithm time complexity:

$$O(V^2)$$

---

13. Theorem for Min-Cut Set

The algorithm outputs a min-cut set with probability at least:

$$\frac{2}{n(n-1)}$$

---

14. Python Simulation

- Students are instructed to open the Campuswire post regarding Lecture 10.
- Download the provided `.ipynb` file from the comments.

End of Lecture 10 Scribe

If you want, I can next:

- Convert this into **exam-answer format**, or
- Reduce it into a **last-day revision sheet**, or
- Add **diagram-based explanations** strictly referencing slide figures only.